

Label	Design Requirement Description
COUNTER_0	rst_n is deasserted most of the time. load_n is asserted 70% of the time. ce is active 70% of the time.
COUNTER_1	When load is asserted and reset is inactive, count_out should take the value of data_load.
COUNTER_2	When reset and load are inactive, count_out increments each cycle while ce=1 and up_down=1.
COUNTER_3	When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is low
COUNTER_4	When reset and load are inactive, count_out decrements each cycle while ce=1 and up_down=0.
COUNTER_5	When counting down, the counter wraps from 0 back to max_count (underflow).
COUNTER_6	When reset is asserted, count_out should be 0, zero flag should be 1, and max_count flag should be 0.
COUNTER_7	All counter operations (reset, load, up-count, down-count) function correctly across 500 random stimulus cycles.

Stimulus Generation	Functional Coverage
Randomization under constraints: rst_n dist {0:5, 1:95}, load_n dist {0:70, 1:30}, ce dist {1:70, 0:30}.	-
Randomization under constraints driving load_n=0 and rst_n=1.	cp1: coverpoint on data_load sampled iff(!load_n && rst_n) — covers all 16 possible load values.
Randomization for up_down and ce, with ce active 70% of the time.	cp2: coverpoint on count_out iff(rst_n && ce && up_down) with individual bins for all values 0 to max_count.
Randomization for up_down, and randomization for enable to be 70% of the time	cp3: transition bin (max_count => 0) on count_out iff(rst_n && ce && up_down).
Randomization for up_down and ce, with ce active 70% of the time.	cp4: coverpoint on count_out iff(rst_n && ce && !up_down) with individual bins for all values 0 to max_count.
Randomization for up_down=0 and ce=1, running 500 times.	cp5: transition bin (0 => max_count) on count_out iff(rst_n && ce && !up_down).
Directed stimulus at start of simulation: rst_n forced to 0 via assert_reset task before any randomization.	-
repeat(500) loop calling Obj.randomize() each iteration, driving all DUT inputs from randomized class fields.	-

Functionality Check
A checker in the testbench to make sure the output is correct in comparison with a golden model.
A checker in the testbench to make sure the output is correct in comparison with a golden model.
A checker in the testbench to make sure the output is correct in comparison with a golden model.
A checker in the testbench to make sure the output is correct in comparison with a golden model.
A checker in the testbench to make sure the output is correct in comparison with a golden model.
A checker in the testbench to make sure the output is correct in comparison with a golden model.
A checker in the testbench to make sure the output is correct in comparison with a golden model.
A checker in the testbench to make sure the output is correct in comparison with a golden model.